



~/term-sudoku \$./run

終端數獨 TermSudoku

可玩 · 可解 · 會自己出題的文字介面數獨

淡江大學資訊管理學系 · Java 程式設計 · 期末分組作品 (G907)

組員 **李佳勳** 414631225 · **黃建瑋** 414630326 · **陳曼婷** 414630086

動機

「用純文字終端機，能不能做出一個會自己出題、而且保證每題只有一個答案的數獨？」

- ▶ 數獨規則單純、邏輯明確； 9×9 盤面用純文字終端機就能完整呈現，契合「文字介面」限制。
- ▶ 一個應用自然涵蓋本學期六大主題（封裝合成／收藏／字串／遞迴／搜索／排序），不需勉強拼湊。
- ▶ 同時具「可玩」與「可解」兩種模式，互動明確、結果可驗證，便於演示與互評。

劇透：能不能「自己出題」？第 6 頁見真章 →

5		3
	9	
8		1

系統概覽 · 畫面設計

```

TermSudoku - java
=====
終端數獨 TermSudoku v1.0
文字介面數獨：可玩 · 可解 · 可評難度
=====

[1] 開始新遊戲（選擇難度）
[2] 從題庫選題遊玩
[3] 自動求解示範（觀看回溯解題）
[4] 難度排行榜
[0] 離開

請輸入選項：

題目 #1（難度：簡單，提示數 32） 已用時間 00:00
※ 輸入「列 行 數」填數（例如 3 5 7），數字填 0 清除；輸入 ? 看完整指令。

  1 2 3   4 5 6   7 8 9
1  . . 3 | . 2 . | 6 . .
2  9 . . | 3 . 5 | . . 1
3  . . 1 | 8 . 6 | 4 . .
   -----
4  . . 8 | 1 . 2 | 9 . .
5  7 . . | . . . | . . 8
6  . . 6 | 7 . 8 | 2 . .
   -----
7  . . 2 | 6 . 9 | 5 . .
8  8 . . | 2 . 3 | . . 9
9  . . 5 | . 1 . | 3 . .

指令： 列 行 數（例 3 5 7） | 清除 列 行 0（例 3 5 0） | 提示 h | 自動解 s
      復原 u | 重來 r | 換題 n | 說明 ? | 返回 q
>

```

終端機文字介面前端

- ▶ **主選單 5 功能**：開始新遊戲（選難度）／題庫選題／自動求解示範／分難度排行榜／隨機出新題
- ▶ 遊玩畫面：列號、行號與 3×3 框線，空格以半形句點標示，狀態列即時顯示剩餘格數與用時。
- ▶ 盤面上方固定「狀態列」顯示上一步結果；底部為「指令列」。

互動 ① 操作與即時驗證

● LIVE DEMO

列 行 數 填入數字 1-9 (例 3 5 7)

列 行 0 清除該格

h / s 提示一格 / 自動求解

u / r / n 復原 / 重來 / 換題

? / q 說明 / 返回

即時驗證

每次填數立刻檢查同列／同行／同宮衝突；題目給定格不可更動，狀態列即時回饋。數字可用全形或半形。

```
TermSudoku - java
題目 #1 (難度：簡單，提示數 32) 已用時間 00:00
※ 第 2 列第 2 行填 9，與同列／同行／同宮衝突。

  1 2 3   4 5 6   7 8 9
1  4 . 3   . 2 .   6 . .
2  9 . .   3 . 5   . . 1
3  . . 1   8 . 6   4 . .
4  . . 8   1 . 2   9 . .
5  7 . .   . . .   . . 8
6  . . 6   7 . 8   2 . .
7  . . 2   6 . 9   5 . .
8  8 . .   2 . 3   . . 9
9  . . 5   . 1 .   3 . .

指令：列 行 數 (例 3 5 7) | 清除 列 行 0 (例 3 5 0) | 提示 h | 自動解 s
      復原 u | 重來 r | 換題 n | 說明 ? | 返回 q
>
```

互動 ② 自動求解 + 分難度排行榜

● LIVE DEMO

TermSudoku - java

自動求解（回溯 backtracking）結果：

	1	2	3	4	5	6	7	8	9
1	4	8	3	9	2	1	6	5	7
2	9	6	7	3	4	5	8	2	1
3	2	5	1	8	7	6	4	9	3
4	5	4	8	1	3	2	9	7	6
5	7	2	9	5	6	4	1	3	8
6	1	3	6	7	9	8	2	4	5
7	3	7	2	6	8	9	5	1	4
8	8	1	4	2	5	3	7	6	9
9	6	9	5	4	1	7	3	8	2

已用回溯法自動求解完成（耗時 0.33 ms）。

一鍵以回溯法（backtracking）自動求解，展示遞迴。
demo：自動解 s → 看排行榜 [4]

TermSudoku - java

【簡單】用時最短前 10 名：

1. 李佳勳	01:13	提示 0 次	2026-05-28
2. 陳曼婷	01:35	提示 1 次	2026-05-29
3. 黃建瑋	01:58	提示 2 次	2026-05-30

【普通】用時最短前 10 名：

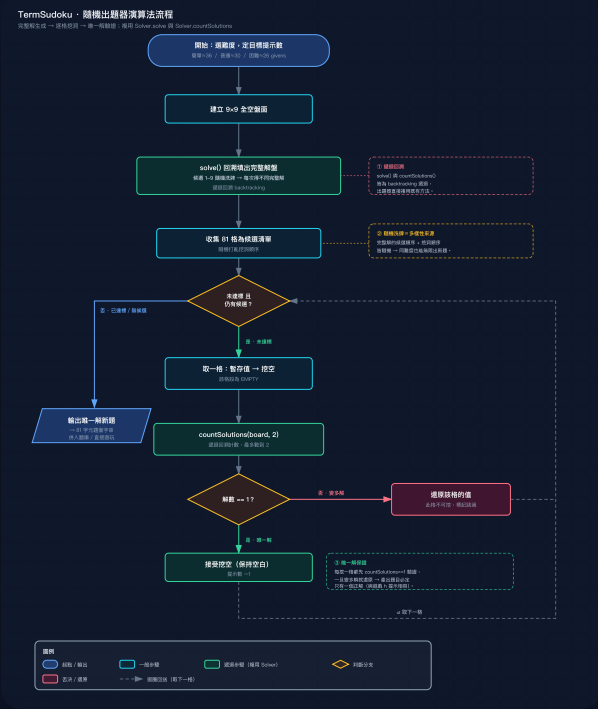
1. 陳曼婷	02:22	提示 0 次	2026-05-29
2. 李佳勳	02:40	提示 1 次	2026-05-31
3. 黃建瑋	03:25	提示 3 次	2026-05-30

【困難】用時最短前 10 名：

1. 李佳勳	05:05	提示 2 次	2026-06-01
2. 黃建瑋	06:28	提示 5 次	2026-05-31

技術亮點 · 隨機出題器

LIVE DEMO



題庫不再只有固定 6 題 —— 程式即時生成全新題目

- 1 完整解隨機生成：回溯填盤時把候選 1-9 洗牌，每次得到不同解盤（遞迴）。
- 2 逐格隨機挖洞：每挖一格用 `countSolutions` 計算解數（遞迴）。
- 3 難度控制：挖到提示數達標（簡單 36／普通 30／困難 26）為止。

唯一解保證

解數 $\neq 1$ 就還原該格 → 產出題目恰有一個正解，與提示 `h`、自動求解 `s` 完全相容，不會生出無解或多解的爛題。



對應本學期六大主題

剛才的出題器，其實同時用到課程一半的主題 —— 整個專案把六大主題都落地：

主題	在程式中的實作
物件封裝合成	Board／Puzzle 私有欄位； Game 組合 Solver／PuzzleBank／Generator／Leaderboard
收藏	PuzzleBank、Leaderboard 的 List ；復原步驟用 Deque
字串	題庫行解析、指令解析、盤面渲染、題面序列化（出題器）
遞迴	回溯求解、解數計數 countSolutions 、出題器 fillRandom 、合併排序
搜索	取難度題走 firstByLevel 二分搜尋 、byId 線性搜尋、回溯搜索
排序	合併排序（依難度）、排行榜依用時排序



心得建議

陳曼婷 414630086

整體來說我覺得這門課對我很有幫助。教授教材準備豐富，有連結、動畫跟 PPT，例如排序動畫用長條圖來呈現，並展現出每種排序的原理，把抽象的過程具象化，讓我更容易理解。此外，教授會清楚解釋每個程式碼的用法，易混淆處也會再多做說明，並在課堂上實際執行程式碼給我們看，這些都讓我覺得這門課真的很實用。

黃建瑋 414630326

透過這次的專題報告，我們將上下學期所學的程式設計與相關概念加以整合並實際應用。由於組內有成員平時喜歡玩數獨，因此我們從一開始便將主題設定為數獨遊戲的開發，希望結合興趣與課堂所學，提升作品的完整性。製作過程中遇到不少挑戰：由於希望盡可能融入課程學到的各項技術與概念，開發初期出現許多錯誤與無法執行的程式碼；面對這些問題，我們透過查閱資料、參考相關做法、反覆測試與除錯，逐步找出問題並加以修正，也結合 AI 工具來優化內容。

李佳勳 414631225 （草稿，待修訂）

這次專題我主要負責程式實作。一開始把功能拆成 Board、Solver、PuzzleBank 等類別時，不太懂為什麼要分這麼細；後來要加排行榜和出題器，發現幾乎不用動到舊程式，才真的體會封裝和合成的好處。最難的是回溯遞迴，自動求解和出題器都靠它，我卡了很久，靠著把過程一步步印出來、自己在紙上跟著跑，才慢慢看懂它怎麼嘗試跟退回。中間也踩了不少 bug，像排行榜讀到壞掉的資料會整個清空，都是反覆測試才抓到。比起只看課本，親手把一個完整程式做出來讓我學到最多。

GitHub 公開倉庫 & 執行方式

倉庫 github.com/nicktodj-boop/TermSudoku

```
$ javac -encoding UTF-8 -d out src/*.java  
$ java -cp out TermSudoku
```

- ▶ 需 JDK 17 以上（開發環境 JDK 25）；在專案根目錄執行（讀 resources/puzzles.txt）。
- ▶ NetBeans：**File → Open Project** 選此資料夾，按 Run 即可，主類別 TermSudoku。
- ▶ 倉庫含全部原始碼、題庫資源與自我測試（test/SelfTest.java）。

終端裡的完整數獨：可玩 · 可解 · 會出題。